20 Web API Interview Question

1. What is a Web API, and why is it used?

A Web API (Application Programming Interface) allows communication between client and server over HTTP. It exposes endpoints for CRUD operations and is commonly used in web/mobile apps to interact with backend services.

---

2. Explain the difference between REST and SOAP APIs.

- **REST**: Lightweight, stateless, uses HTTP verbs, and returns JSON or XML. Easier to integrate.

- **SOAP**: Protocol-based, XML-only, has strict standards and built-in security (WS-Security).

---

3. How does authentication work in Web APIs?

Authentication verifies a user's identity. Common methods:

- **Basic Auth**
- **Token-based (JWT, OAuth)**
- **API Keys**

---

4. What is CORS, and why is it important?

CORS (Cross-Origin Resource Sharing) is a browser mechanism that restricts web pages from making requests to a different domain. It must be configured on the server to allow cross-origin requests.

5. How do you handle rate limiting in Web APIs?

Rate limiting restricts the number of requests a client can make in a given time frame. Techniques:

- Token bucket
- Leaky bucket
- Throttling policies in API Gateways

---

6. What is an API Gateway, and what role does it play?

An API Gateway manages and routes client requests to backend services. It handles:

- Routing
- Authentication
- Rate limiting

- Aggregation
- Monitoring

---

7. Explain token-based authentication in Web APIs.

After successful login, the server issues a token (like JWT). Clients include the token in headers for subsequent requests. Server validates the token to authenticate the user.

---

8. What is Swagger/OpenAPI, and how is it used?

Swagger/OpenAPI is a specification for documenting APIs. It allows:

- API exploration
- Auto-generated docs
- Client SDK generation
- Testing endpoints

9. How do you secure a Web API?

- Use HTTPS
- Implement authentication/authorization
- Input validation
- CORS configuration
- Rate limiting
- Security headers (e.g., CSP, HSTS)

---

10. What is dependency injection in Web API development?

Dependency Injection (DI) is a design pattern where dependencies (services, repositories) are injected into a class rather than being created internally. Promotes testability and loose coupling.

---

11. How does versioning work in Web APIs?

Common versioning strategies:

- URL (e.g., /api/v1/products)
- Query string (e.g., ?version=1)
- Header (e.g., Accept: application/vnd.company.v1+json)

---

12. Explain middleware in ASP.NET Web API.

Middleware components process HTTP requests and responses. They can be chained together to add features like logging, authentication, and error handling.

---

13. What is the difference between synchronous and asynchronous API calls?

- **Synchronous**: Blocks execution until the task is complete.
- **Asynchronous**: Frees up the thread, allowing other tasks to run. Improves scalability and performance.

---

14. How do you implement logging in Web APIs?

Use logging libraries like Serilog, NLog, or built-in ASP.NET Core logging. Log request/response data, errors, and custom events for diagnostics and monitoring.

---

15. What is content negotiation in Web APIs?

It's the process of selecting the appropriate response format (JSON, XML, etc.) based on the Accept header sent by the client.

16. How do you handle exceptions globally in Web APIs?

Use:

- **Exception filters** (IExceptionFilter)
- **Middleware** for centralized error handling
- Custom error responses/logging

17. What is the difference between IHttpActionResult and HttpResponseMessage?

- IHttpActionResult: Introduced in Web API 2; promotes testability and abstraction.
- HttpResponseMessage: Gives full control over the HTTP response.

18. How does attribute routing work in Web APIs?

You decorate controllers and actions with [Route("path")] attributes for defining custom routes instead of relying on default routing conventions.

---

19. What is model binding in Web APIs?

Model binding maps incoming HTTP request data to parameters or objects in controller methods automatically (from body, query string, route, etc.).

---

20. How do you implement file upload in Web APIs?

Use IFormFile or MultipartFormDataContent in .NET Core. The server reads file data from the request and stores it as needed.